Database Management system

Page No. | Date

A database is base for data that stores a collection of data. Each database has one or more distinct APIs for creating, accessing, managing, searching and replicating data it holds.

Database is computerised record keeping system.

A Database Management system (DBMS) is collection of interrelated data and set of programs to access those data.

Application ⟷ DBMS ⟷ O.S ⟷ Database.

- DBMS also provides safety for stored information from data corruption and or unauthorised access.

Few terms

(i) Application programmer → develop application program. These program can manipulate database in all possible ways.

(ii) End user → access database from a terminal using query language provided by database system or through application program.

(iii) DBA (Database Administrator) → is person who is responsible for design, construction and maintenance of database.

	DBMS	File Processing system.
(i)	No data redundancy	(i) data redundancy
(ii)	No data inconsistency	(ii) Data inconsistent
(iii)	accessing data is easier	(iii) data is scattered in various files.
(iv)	data is secure	(iv) data is not secure.
(v)	concurrent access and crash recovery	(v) no concurrent access and no recovery.
(vi)	Atomicity & integrating problem not found	(vi) Here it exists.

Page No.
Date

Database system and Architecture.

↳ Internal level

(i) **Physical level**: Lowest level of abstraction and describes how data are actually stored.
- complex low-level data structure.

↳ conceptual level (Abstraction def^n discussed later)

(ii) **Logical level**: next-higher level of abstraction and describes what data are stored in database, relationship among those data.
- relatively simple structure (global view)

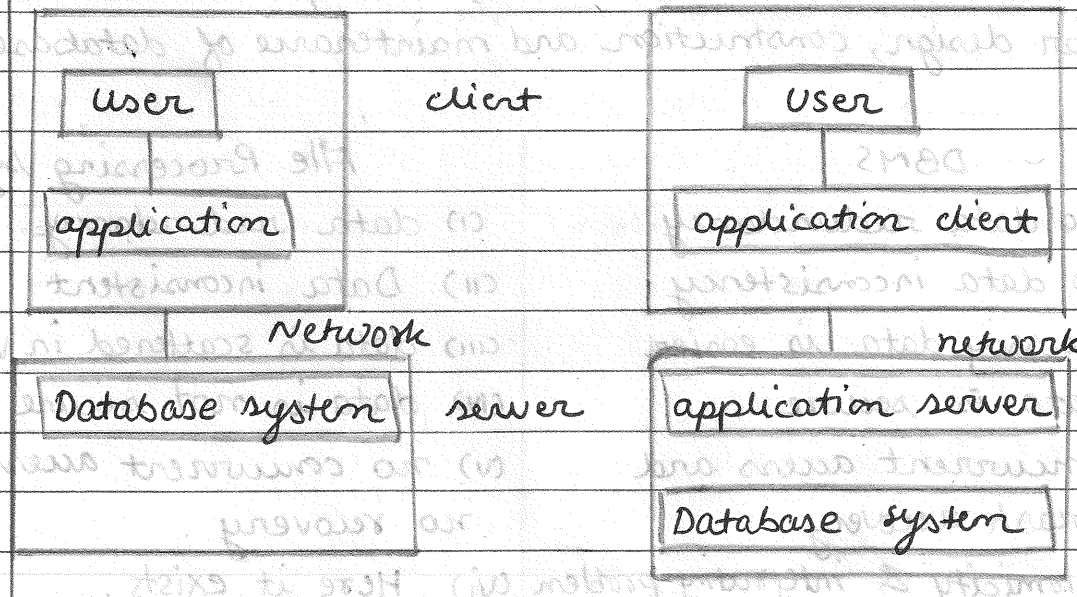
↳ External level.

(iii) **View level**: highest level of abstraction and describes only part of entire database.
- uses simpler structure, complexity remains because of variety of information stored in large database.
- system may provide many views of same database.

View level
Logical
Physical

Application architecture of DBMS.

Two-tier architecture | Three-tier architecture.



2-tier architecture | 3-tier architecture.

Page No.
Date

Two tier architecture

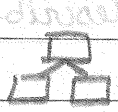
- application is partitioned into component that resides at client machine, which invokes database system functionality at server machine through query language.
- application program interface standard like ODBC (open database connectivity) and JDBC (Java database connectivity) are used for interaction b/w client & server.

Three tier architecture.

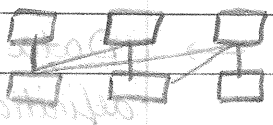
- client machine acts as merely as front end and doesnot contain any direct database calls.
- application server in turn communicates with database system to access data.

Three tier application are more appropriate for large applications and for applications that run on WWW.

Database Models

(i) Hierarchical Database Model. 

- oldest form of database developed by IMS.
- organises data in tree structure. (one to many relationship).

(ii) Network Database Model. 

- provides multiple paths b/w segments.
- 1:1, 1:M, M:M relationship

(iii) Relational Database Model.

- represent data as two-dimensional table
- built up relations.

(iv) Object Oriented Database Model.

- user can define own data access methods, representation of data and method of manipulating it.
- an object is an item that can contain both data and procedure that manipulates data.
- can store more data & access them faster.
- query language used → object Query Language (OQL).

→ Schema & Instances.

The collection of information stored in database at a particular moments is called an instance of database.

The overall design of database is called database schema.

There are three types of schema.

(i) Physical Schema: describes database design at physical level. It specifies additional storage details.

(ii) Conceptual schema: (logical schema) describes stored data. It describes all relations that are stored in database.

(iii) External schema: allows data access to be customised at level of individual user or group of users.

→ Data Independence: ability to modify a schema definition in one level without affecting a schema definition in next higher level.

- Physical data Independence (seperation of conceptual view from internal view)
- Logical Data Independence. (seperation of external view from conceptual view)

Page No.
Date

Logical data independence is difficult to achieve than physical data independence because application programs are very much dependent on logical structure of data.

Many database system users are not computer trained, so developers hide complexity from users through several level of abstraction, to simply user's interaction with system. This concept is known as data abstraction.

RDMS was developed by Dr. E.F. Codd of IBM

Popular RDBMS are Oracle, SQL server, MYSQL, PostgreSQL etc.

(i) Relation: is a table with column and rows, which represent data items and relationship among them.
- Name → title or entity identifier
- Cardinality → no. of tuples (rows) in a relation.
  - or (max no. of relationship an entity can participate.
- Degree → no. of attributes (columns) in each tuple.
  - or (how many entity sets are participating.

Participation → min no. of relationship an entity can participate (existence)

both cardinality & participation are called structional constraint.

(ii) Domain: collection of all possible values from which values for a given column (attribute) is drawn. A domain is said to be atomic if elements are considered to be individual unit
eg: Book_id

(iii) Attributes: heading column of a table relation are known as attribute. Each attribute of a table has a distinct name. Also called fields.

(iv) Tuples: rows in a relation is called tuple. Each row or tuple has a set of permitted values for each attribute.

attributes

Book_id	Author_id	Emp_Salary	Years.
10	Khushi	5000	1
11	Ramesh	15000	2
12	Kavya	10000	3

tuples

→ Data

- User data: contains a table of data in form of rows and column
- Meta data: data that describes properties or keep information of other data.
- Application data: contains structure and format of queries, reports & application components.

→ keys.

Integrity of information stored in database is controlled by keys. A key or key field is a column value in a table that is used to either uniquely identify a row of data in a table, or establish a relationship with another table.

(i) Super key: is a set of one or more attribute that taken collectively, allow us to identify uniquely an entity in entity set.

eg:

Roll no.	Name	Branch	DOB	Year.

Page No. | Date

Super key$_1$ = { Rollno, Name, Branch }
Super key$_2$ = { Name, Branch, DOB }
super key$_3$ = { Name, DOB, Year }

(ii) Candidate key. (minimal superkey)
- Combination of one or more fields whose values uniquely identify a record in a table.

Candidate key$_1$ = { Rollno, Name }
candidate key$_2$ = { Name, DOB }
candidate key$_3$ = { Name, Branch }

(iii) Primary key
↳ should uniquely identify each entity. and value should not be null.

Primary key = Unique + Not Null.

eg: {Roll no}.

(iv) Composite key.
- is a key composed of more than one column.
- sometimes called concaterated key / structured key.

(v) Foreign key.
- combination of one or more columns in a table (parent table) at references a primary key in another table (child table).
- foreign key value is derived from primary key of another table.

(vi) Alternate key
- any candidate key other than the one chosen as primary key

Page No.
Date

SQL (Structured Query Language)
- is non-procedural computer language aimed to store, manipulate and query data stored in relational database.

Data types.
- is a data storage format that can contain a specific type or range of values.

(i) Numeric data types
↳ int → integer (4 bytes)
↳ float (N,D) → floating decimal point (4 bytes)
↳ double (N,D) → larger no. floating decimal point (8 bytes)
↳ decimal (N,D) → decimal

(ii) String / Text data types
↳ char → (0 to 255)
↳ varchar → variable + character

(iii) Date and time data types.
↳ DATETIME → both date and time (YYYY-MM-DD HH:MM:SS)
↳ DATE → only date (YYYY-MM-DD)
↳ TIMESTAMP → values are converted from current time zone to UTC (co-ordinated Universal Time) while storing and viceversa converting. (YYYY-MM-DD HH:MM:SS)

Page No.
Date

SQL statement

- SQL stament starts with : select, insert, update, delete, alter, drop, create, use, show
- ends with semicolon (;)

DDL (Data Definition language)

- used to create, modify and delete database structure but not data.

The main task of DDL:

(i) To create database objects
(ii) To modify database objects
(iii) To destroy database object

ex: create, alter, drop, rename, truncate are DDL statement.

(a) CREATE TABLE STUDENT // create table student
{

};

(b) DESC STUDENT; or DESCRIBE STUDENT; // describes a table ie description of a table Student.

(c) DROP TABLE emp; // it deletes/drop table completely. ie data + structure both

(d) ALTER TABLE emp RENAME TO employee;
// rename table emp to employee.
ie RENAME old tablename TO newtable name

(e) TRUNCATE TABLE table_name; // delete only data. Structure is not deleted.

Page No.
Date

DML (Data Manipulation Language)

- enables user to access or manipulate data.

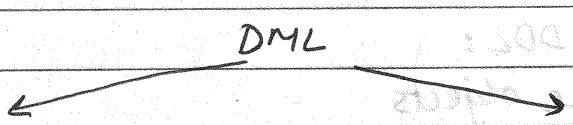
Main task of DML is:

(i) Insert new data in database

(ii) modify data already stored in database

(iii) deletion of data

(iv) retrieve information.



Procedural DML	Non Procedural DML.
- requires user to specify what data are needed and how to get those data.	- requires a user to specify what data are needed without specifying how to get those data.
- requires writing procedures, methods to specify what data is needed	- collection of commands for data manipulation including insert, update, delete and query data.

ex: Insert, Select, Update, Delete.

(i)
```
INSERT INTO CUSTOMERS (ID, NAME, AGE, SALARY)
VALUES (1, 'khushi', 32, 2000.00);
```

OR

```
INSERT INTO CUSTOMERS VALUES (1, 'khushi', 32, 2000.00);
```

(ii)
```
SELECT column1, column2
FROM table
where cond^n;
```

Page No.
Date

asterisk (*) is quick way of selecting all columns.

SQL is not case sensitive.

(iii) UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE cond^n ;

(iv) DELETE FROM table_name // deletes statement one by one.
WHERE cond^n ;

(iii) DCL (Data Control Language)
- used to assign security levels in database which involves multiple user setups.
- used to define role and access privileges to users.

(i) GRANT : command is used to give user's access privileges to database.

GRANT privilege_name ON object_name TO
{ user_name / PUBLIC / role_name } [with GRANT option]

(ii) REVOKE : command is used to withdraw access privileges given with GRAND command.

REVOKE privilege_name ON object_name
FROM { user_name / PUBLIC / role_name }

(IV) TCL Commands.

- used for controlling transaction in database system.
- used to manage changes made by DML

(i) COMMIT - used to save work done.

Syntax: COMMIT;

(ii) SAVEPOINT - used to identify a point in transaction to which you can latter rollback.

Syntax: savepoint_name;

(iii) Rollback - used to restore database to original since last COMMIT.

Syntax - ROLLBACK;

(iv) SET transaction.

Transaction

Collection of operations that form a single logical unit of work is called transaction.

Properties of Transaction (ACID)

(i) Atomicity: transaction is either performed completly or not performed at all.

(ii) Consistency: ensures database is update correctly when multiple user are updating database concurrently.

(iii) Isolation: execution of transaction should not be interfered with by any other transaction being executed concurrently.

(iv) Durability: changes applied to database by a committed transaction must persist in database. These changes must not be lost because of any failure.

Page No.
Date

More Commands.

(i) Where clause

used to extract only those records that fulfill a specified criteria.

Syntax:
```
SELECT column_name(s)
FROM table_name
WHERE condition;
```

(ii) ORDER BY clause.

is used to sort result set along a specified column.

bydefault: sorts records in ascending order.

you want to sort in descending order → use DESC keyword.

syntax:
```
SELECT column_name(s)
FROM table_name
ORDER BY column_name(s) ASC/DESC;
```

(a) sorting in Ascending order.

Table name: company

Sno	Name	order
1	Ravi	Book
2	Akshay	Game
3	Zep	keypad

```
SELECT * FROM company ORDER BY Name DESC;
```

↓

Sno	Name	order
3	Zep	keypad
1	Ravi	Book
2	Akshay	Game

(iii) Sorting By Column Alias

↳ if column alias is declared anywhere in statement then it can also be used even for sorting records.

syntax:

```
SELECT Name as Name_of_client, Order
FROM Company
ORDERBY Name_of_client;        // by default ASC
```

Name_of_client	Order
Akshay	Game
Ravi	Book
Zep	Keypad

(iv) Arithmetic operators.

can be used in SELECT statement to retrieve information from table. Operator → +, −, *, /

Syntax:
```
SELECT <Expression 1> operator <Expression 2>
FROM <tablename>
WHERE <condition>;
```

Null values in Arithmetic Expression.

Column with NULL value can be selected in SELECT query and can be part of an arithmetic expression.

→ Any arithmetic expression using NULL value results in NULL.

eg: SELECT 7 + NULL;

Result: NULL.

Page No. | Date

(v) Comparision Operator. (Relational operator)
used to test or compare value of two operands
(b/w two variable or one variable and a constant)
If condition false, result is 0.
condition true, result is 1.

Operator → =, >, <, >=, <=, <> or != , !<, !>

Syntax: SELECT <column name>
FROM <table_name>
WHERE <expression> comparision operator
<expression> ;

(vi) Logical operator (Boolean operator)
compare two conditions at a time to determine whether a row can be selected for output or not. They return boolean data type value as TRUE or FALSE.

operator → AND, OR, NOT

Syntax: SELECT <column name>
FROM <table_name>
WHERE <expression> boolean operator <expression>;

(v) BETWEEN operator
Selects values within a given range. Value can be numbers, text or dates. (begin and end values are included)

Syntax: SELECT column_name(s)
FROM table_name
WHERE column_name BETWEEN value 1 AND
value 2 ;

Page No.
Date

eg: SELECT *
FROM Products
WHERE Price BETWEEN 10 AND 20;

(vi) IN operator

↳ checks value within a set of values (seperated by commas) : shorthand for "multiple OR condition".

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE column_name IN (value1, ...., valuen);
```

or

```
SELECT column_name(s)
FROM table_name
WHERE column_name IN
(SELECT STATEMENT);
```

eg:

```
SELECT *
FROM Company
WHERE Order IN
('Book', 'Orange', 'Game');
```

or

```
SELECT * FROM CUSTOMERS
WHERE Country IN ( SELECT Country
FROM Suppliers)
```

↓

output: selects all customers that are from same countries as suppliers.



Page No.
Date

(vii) Like Operator

L to search specified pattern in a column.

There are two wildcards used in conjuction with LIKE operator

(i) % (percentage) → represent to match zero, one or multiple characters.

(ii) _ (underscore) → represent single character.

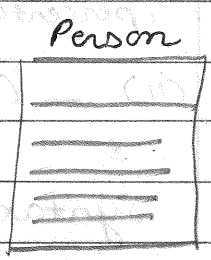
Syntax:
```
SELECT column_name(s)
FROM table_name
WHERE column LIKE pattern;
```

eg: WHERE customer_name	value that starts with
(i) LIKE 'a%'	"a".
(ii) LIKE '%a'	value that ends with "a".
(iii) LIKE '%an%'	value that has "an" in any position.
(iv) LIKE '_a%'	value that has "a" in second position
(v) LIKE 'a_%_%'	value that starts with "a" and are atleast 3 character in length.
(vi) LIKE 'a%o'	value that starts with "a" and ends with "o"

(viii) IS NULL operator.

A null in table column means value is missing. To compare, special operation IS NULL tests a column for null because two null cannot be compared.

eg:
```
SELECT Name, Address
FROM Person
WHERE Address IS NULL;
```



will give output:

Name and address column where address is null.

Name	Address
Akshay	
Mayank	

} address is NULL.

We can use IS NOT NULL as well to get values where address is NOT NULL.

(ix) Concatenation operator (||)

used to join two string values or expression in SELECT query.

only applicable for character and string column values resulting into a new character expression.

Syntax:

```
SELECT "string 1" || "string 2"
FROM table_name.
```

Page No.
Date

(x) Operator Precedence.

+, ~	identity, negation
*, /	multiplication, division
+, -, ||	addition, subtraction, concatenation
=, !=, <, >, <=, >=, ISNULL, LIKE, IN, BETWEEN	comparision
NOT	logical
AND	logical AND
OR	logical OR.

(xi) Some common operators

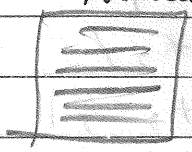
(i) NOT → displays record if any of condition(s) is not true.
(ii) AND → displays record if all condition are true.
(iii) OR → displays record if any condition is true.
(iv) MIN() → returns smallest value of selected column.
(v) MAX() → returns largest value of selected column.

Syntax:
```
SELECT column1, column2...
FROM tablename
WHERE condn1 (column1) AND cond2 (column2) AND cond3 (column3)..;
  or
WHERE cond1 OR cond2 OR cond3... ;
  or
WHERE NOT cond2;
```

Syntax:
```
SELECT MIN (column_name)
FROM tablename WHERE condn ;
```

eg:
```
SELECT MIN(Price) AS smallestPrice
FROM Products;
```

Products.



eg:
```
SELECT *
FROM Customers
WHERE Country = "Germany" AND City = "Berlin";
```

eg:
```
SELECT *
FROM Customers
WHERE NOT Country = "Germany";
```

eg:
```
SELECT *
FROM Customers
WHERE Country = "Germany" AND ( City = "Berlin"
OR City = "Munchen" );
```

eg:
```
SELECT *
FROM Customers
WHERE NOT Country = 'Germany' AND NOT
Country = 'USA';
```

you can use double quotes (") or single quotes (') in string.

Page No.
Date

→ GROUP BY

used with aggregate functions (COUNT, MAX, MIN, SUM, AVG) to group result-set by one or more columns.

Syntax:
```
SELECT columnname(s)
FROM table_name
WHERE cond^n
GROUP BY column_name(s)
ORDER BY column_name(s);
```

eg:
```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country;
```

COUNT(CustomerID)	Country
3	Austria
2	Brazil
2	Canada
9	India
5	Italy

eg:
```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
ORDER BY COUNT(CustomerID) DESC;
```

COUNT(CustomerID)	Country
9	India
5	Italy
3	Austria
2	Brazil
2	Canada

→ HAVING Clause.

↳ was added in SQL because WHERE keyword couldnot be used with aggregate functions.

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
HAVING condition
ORDER BY column_name(s);
```

eg:
```
SELECT COUNT (CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 5;
```

COUNT(CustomerID)	Country
9	Brazil
11	France
11	Germany
7	UK

eg:
```
SELECT COUNT (CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 5
ORDER BY COUNT (CustomerID) DESC;
```

Country (CustomerID)	Country
11	France
11	Germany
9	Brazil
7	UK

Page No.
Date

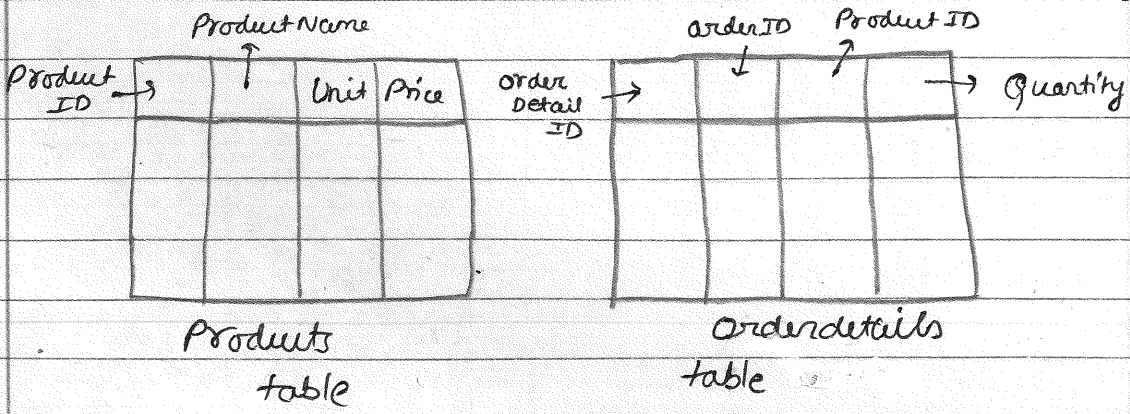
→ ANY and ALL operators.
- used with WHERE or HAVING clause.
- ANY returns true if any of subquery values meets condition
- ALL returns true if all of subquery values meet condition.

Syntax:

```
SELECT column-name(s)
FROM table_name
WHERE column_name operator
ANY/ALL
(SELECT column_name
 FROM table_name
 WHERE cond^n ;
)
```

eg:
```
SELECT Productname
FROM Products
WHERE ProductID = ANY
(SELECT Product ID FROM orderDetails
 WHERE Quantity >99 ) ;
```

→ returns TRUE and lists productnames if it finds ANY records in orderDetails table where quantity >99.



Page No.
Date

Data type conversion Functions

There are two data type conversion as follows:

(a) Implicit Data Type Conversion

↳ this conversion depends on context in which it occurs and may not work same way in every case.

e.g. The text literal '20' has datatype CHAR.
SQL implicitly converts it to number datatype if it appears in numeric expression

```
SELECT salary + '20'
FROM employees;
```

(b) Explicit Data Type Conversion.

↳ are single row functions which are capable of typecasting column value, literal or an function.

TO_CHAR() Function: used to typecast a numeric or data input to character type with format model.

Syntax:

```
TO_CHAR (number 1, [format], [nls-parameter])
```

TO_NUMBER() Function: used to typecast character value to numeric datatype. If string being converted contains non-numeric characters, func^n returns error.

Syntax:

```
TO_NUMBER( string 1 , [format], [nls-parameter]),..
```

TO_DATE() Function: used to enter a date in any format and then it convert the entery into default format used by SQL.

Syntax: `TO_DATE (String 1, [format_mask], [nls_language])`